

Université Internationale de Rabat

THE INNOVATIVE UNIVERSITY

Hands-on on Quantum Computing using Qiskit

Installing Qiskit and getting started

2026-2027

I Lab's objective

Throughout this lab, we will install and configure Qiskit, an open-source framework developed by IBM to implement and simulate quantum circuits. It also provides a user-friendly and versatile platform that allows exploring the capabilities of quantum computing. After completion of this lab, you will successfully be empowered to take the first steps into quantum computing programming.

II Setting-up the Environment on Ubuntu

In what follows, we are going to install required tools.

1. Make sure you have python3 command available.

   ```
   sudo apt update; sudo apt install python3
   ```

2. Create a directory and run the following command, you may need to install python3.10-venv package.

   ```
   mkdir qiskit-project; python3 -m venv qiskit-project/
   ```

3. Activate your new environment. Your terminal prompt should be prefixed by "(qiskit-project)".

   ```
   source qiskit-project/bin/activate
   ```

4. Use pip to install qiskit and then install qiskit runtime. You can run "pip list" to see active packages in your virtual environment.

   ```
   pip install qiskit; pip install qiskit-ibm-runtime; pip list
   ```

5. If you intend to use visualization functionality or Jupyter notebooks, it is recommended to install Qiskit with the extra visualization support ('qiskit[visualization]').

   ```
   pip install qiskit[visualization]
   ```

6. If you want to run a Jupyter notebook with the Qiskit packages you just installed, you need to install Jupyter in your environment.

   ```
   pip install jupyter
   ```

7. Then open your notebook as follows :

   ```
   jupyter notebook
   ```

8. Congrats. Your installation is done, especially if you are planning to work locally and use simulators built into Qiskit.

III First Qiskit Program : 1-qubit

Set-up a simple example : creating a quantum circuit with one qubit and measuring it.

1. Enable inline plotting for Jupyter Notebook.

```
%matplotlib inline

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import (
    plot_histogram,
    plot_bloch_multivector,
    plot_state_qsphere
)
from matplotlib import pyplot as plt
from IPython.display import display
```

2. Create Quantum Circuit and apply Hadamard Gate (superposition)

```
qc = QuantumCircuit(1, 1)

qc.h(0)
```

3. Quantum Circuit, Statevector, and Bloch Sphere/Q-Sphere visualization (Before Measurement, Fig. 1)

```
display(qc.draw("mpl"))  # Draw circuit diagram


state = Statevector.from_instruction(qc)

print(state) # print Statevector
```

```
plot_bloch_multivector(state)

plt.title("Bloch Sphere of |+> State")

plt.show()

plot_state_qsphere(state)

plt.title("Q-Sphere Representation")

plt.show()
```

It is worth noting that a single qubit quantum state can be written as :

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where

— $|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}, \quad |1\rangle = \begin{bmatrix}0\\1\end{bmatrix}$,

— $\alpha, \beta \in \mathbb{C}$ are the complex probability amplitudes,

— $|\alpha|^2$ and $|\beta|^2$ give the probabilities of measuring 0 and 1, respectively.

For the given statevector :

`Statevector([0.70710678+0.j, 0.70710678+0.j])`,

we have :

$$\alpha = 0.70710678 + 0.j \approx \frac{1}{\sqrt{2}}, \quad \beta = 0.70710678 + 0.j \approx \frac{1}{\sqrt{2}}.$$

Thus, the qubit state is :

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle.$$

In summary,

— This qubit is in an **equal superposition** of $|0\rangle$ and $|1\rangle$.

— The probability of measuring 0 is $|\alpha|^2 = \left(\frac{1}{\sqrt{2}}\right)^2 = 0.5$.

— The probability of measuring 1 is $|\beta|^2 = \left(\frac{1}{\sqrt{2}}\right)^2 = 0.5$.

— The $+0.j$ indicates there is no imaginary component ; the amplitudes are purely real.

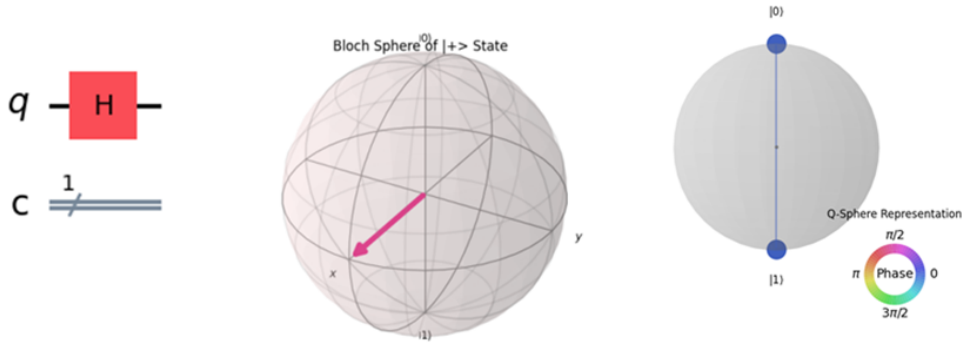


FIGURE 1 – Representation

4. Add measurement, simulation and measurement probability distribution, Fig. 2

```
qc.measure(0, 0)

display(qc.draw("mpl"))

simulator = AerSimulator()
job = simulator.run(qc, shots=1024)
result = job.result()
counts = result.get_counts()

print(counts)

plot_histogram(counts)
plt.title("Measurement Probability Distribution")
plt.show()
```

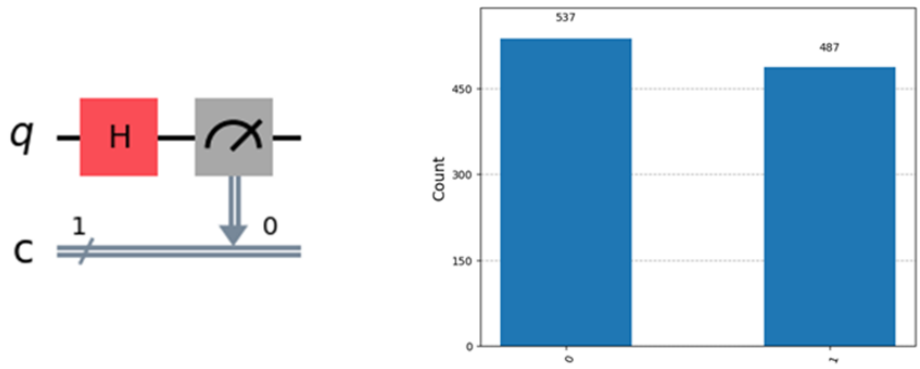


FIGURE 2 – Measurement

1. Starting from the computational basis states $|0\rangle$ and $|1\rangle$, modify the circuit (qiskit) to obtain the following (Figure 3). For each case, explain which initial state is chosen and which gates are necessary. There is no rule, identify what is missing in the target state : superposition, sign change, or phase. Then select the gates that produce that effect.
   — $|-\rangle$, is at the point (-1, 0, 0), on the -x-axis
   — $|i\rangle$, is at the point (0, 1, 0), on the -y-axis
   — $|-i\rangle$, is at the point (0, -1, 0), on the -y-axis
   — Hadamard gate (H) : creates superposition

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

   — Pauli-X gate (X) : flips $|0\rangle \leftrightarrow |1\rangle$

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

   — Pauli-Z gate (Z) : flips the sign of $|1\rangle$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

   — Phase gate (S) : adds a phase i to $|1\rangle$

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

   — Adjoint Phase gate (S†) : adds a phase $-i$ to $|1\rangle$

$$S^\dagger = \begin{pmatrix} 1 & 0 \\ 0 & -i \end{pmatrix}$$

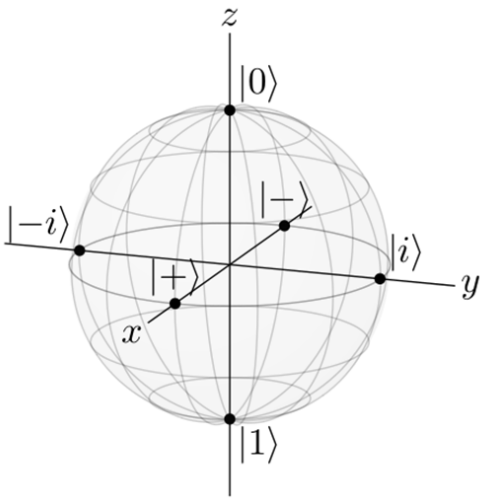


FIGURE 3 – Superposition states on the Bloch sphere. A qubit can be any point on the Bloch sphere.

2. For each of the following states $|+\rangle$, $|-\rangle$, $|i\rangle$, $|-i\rangle$, determine the spherical coordinates (θ, ϕ) on the Bloch sphere by comparing each state with the general form

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle.$$

Use Euler's relation to identify the phase factor when needed.

$$e^{i\phi} = \cos\phi + i\sin\phi$$

Check each state in Qiskit and verify its position using the Bloch sphere representation. You can also determine the (x, y, z) coordinates (Fig. 4).

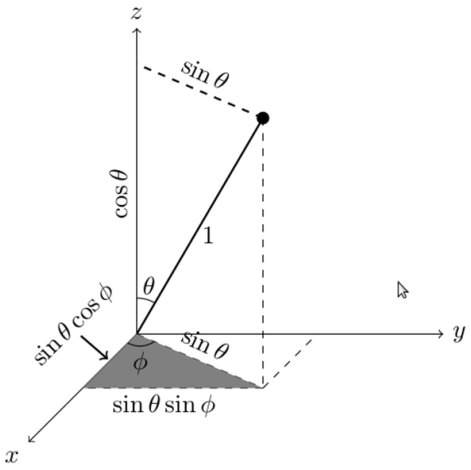


FIGURE 4 – Cartesian coordinates.

IV Second Qiskit Program : 2-qubit

This example demonstrates how quantum superposition and entanglement are created using a simple two-qubit circuit. If we prepare two qubits both in the $|+\rangle$ state, we compute the tensor product :

$$|\psi\rangle = |+\rangle \otimes |+\rangle$$

Substituting the definition of $|+\rangle$:

$$|\psi\rangle = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) \otimes \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)$$

Expanding the product :

$$|\psi\rangle = \frac{1}{2}\left(|00\rangle + |01\rangle + |10\rangle + |11\rangle\right)$$

This is a superposition of all four computational basis states, each with equal probability :

$$P(00) = P(01) = P(10) = P(11) = \frac{1}{4}$$

An entangled Bell state is defined as :

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right)$$

In the Bell state :

$$P(00) = \frac{1}{2}, \qquad P(11) = \frac{1}{2}$$

The outcomes 01 and 10 never occur.

Below are main steps, see Ex2.

1. Creating the Circuit

```
qc = QuantumCircuit(2, 2)
```

This line creates a quantum circuit with 2 quantum qubits and 2 classical bits (for storing measurement results). Initially, both qubits are in the default state : $|00\rangle$.

2. Applying the Hadamard Gate

```
qc.h(0)
```

The Hadamard gate is applied to qubit 0. It transforms :

$$|0\rangle \to \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Since only the first qubit is modified, the two-qubit state becomes :

$$|00\rangle \to \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$$

At this stage :
- The first qubit is in superposition.
- The second qubit is still in state $|0\rangle$.
- The qubits are not yet entangled.

3. Applying the CNOT Gate

```
qc.cx(0, 1)
```

The Controlled-NOT (CNOT) gate uses :
- Qubit 0 as the control
- Qubit 1 as the target

Rule of CNOT :
- If the control qubit is $|0\rangle$, do nothing.
- If the control qubit is $|1\rangle$, flip the target qubit.

Applying CNOT to the superposition :

$$\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$$

The $|00\rangle$ term remains unchanged. The $|10\rangle$ term becomes $|11\rangle$ (because the target qubit flips). Therefore, the final state is :

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

4. Result : Bell State This state is called a Bell state :

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Now the qubits are entangled :

— Measuring one qubit immediately determines the other.

— The state cannot be separated into two independent single-qubit states.

As depicted in Fig. 5, the + symbol in a circuit diagram is part of the CNOT (CX) gate notation. It indicates the target qubit of the CNOT gate. The other qubit, connected by a vertical line to the +, is the control qubit, usually marked with a solid dot. In fact, after applying a Hadamard gate to qubit q_0, the two-qubit state becomes :

$$\frac{|00\rangle + |10\rangle}{\sqrt{2}}$$

Now the CNOT gate is applied with q_0 as the control and q_1 as the target :

— For $|00\rangle$: $q_0 = 0 \rightarrow$ no flip $\rightarrow$ remains $|00\rangle$.

— For $|10\rangle$: $q_0 = 1 \rightarrow$ flip $q_1 \rightarrow$ becomes $|11\rangle$.

The final state is the Bell state :

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

This shows that the qubits are now entangled : measuring one immediately determines the other.

— The state is a superposition of $|00\rangle$ and $|11\rangle$ only.

— If we measure the first qubit and get 0, the only compatible term in the superposition is $|00\rangle$. Therefore, the second qubit must also be 0.

— If we measure the first qubit and get 1, the only compatible term is $|11\rangle$. Therefore, the second qubit must also be 1.

In summary, entanglement and superposition are the backbone of quantum algorithms. Bell states are a simple example of entanglement, but the same principles scale to larger multi-qubit systems, enabling exponential speed-ups over classical algorithms in tasks like search, factorization, and secure communication.

— Superposition allows parallelism : qubits can explore many states at once.
— Entanglement allows correlations : outcomes of one qubit affect others in predictable ways.
— Together, they are why quantum algorithms can outperform classical ones.

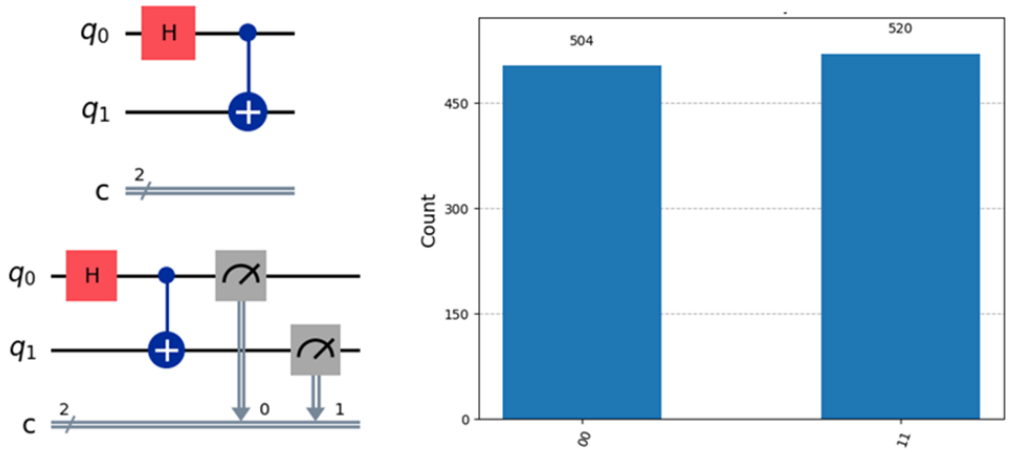


FIGURE 5 – Circuit and measurement

V Third Qiskit Program : Multi-Qubit

The objective is to extend the 2-qubit entanglement (Bell states) to 3 or more qubits.

— Create multi-qubit superposition and entangled states.
— Observe measurement correlations and visualize the states.

The 3-qubit GHZ state is defined as :

$$|\text{GHZ}_3\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

1. Initialize 3 qubits in $|000\rangle$.
2. Apply a Hadamard gate to qubit 0 to create superposition :

$$|000\rangle \xrightarrow{H_0} \frac{|000\rangle + |100\rangle}{\sqrt{2}}$$

3. Apply a CNOT gate with qubit 0 as control and qubit 1 as target :

$$\frac{|000\rangle + |100\rangle}{\sqrt{2}} \xrightarrow{CNOT_{0\to1}} \frac{|000\rangle + |110\rangle}{\sqrt{2}}$$

4. Apply a CNOT gate with qubit 0 as control and qubit 2 as target :

$$\frac{|000\rangle + |110\rangle}{\sqrt{2}} \xrightarrow{CNOT_{0\to2}} \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

As shown in Fig. 6, all 3 qubits in the computational basis are measured.

— Only the outcomes $|000\rangle$ and $|111\rangle$ are possible.
— Probabilities : $P(000) = \frac{1}{2}, \quad P(111) = \frac{1}{2}$
— Outcomes $|001\rangle$, $|010\rangle$, $|011\rangle$, etc., never occur.
— Measuring any one qubit immediately determines the others due to entanglement.

It is worth noting that a GHZ (Greenberger-Horne-Zeilinger) state is a type of entangled state that generalizes Bell states to three or more qubits. The canonical 3-qubit GHZ state is defined as :

$$|\text{GHZ}_3\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

For n qubits, the GHZ state is :

$$|\text{GHZ}_n\rangle = \frac{|0\rangle^{\otimes n} + |1\rangle^{\otimes n}}{\sqrt{2}}$$

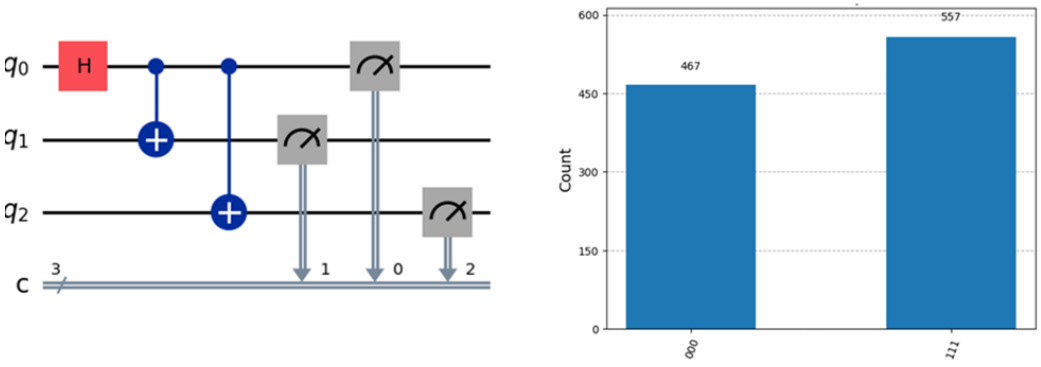


FIGURE 6 – Circuit and measurement